

# Manipulação de erros

O código Dart pode lançar (throw) ou pegar (catch) exceções ou erros.

```
void main() {  
    throw 'Erro!';  
}
```

# Throw

Quando acontecer uma exceção:

```
void main() {  
    throw FormatException('Faltou um argumento!');  
}
```

# try

Podemos usar `try...on` para executar algo que pode gerar um erro:

```
void main() {  
  try {  
    adicionarTarefa();  
  } on SemTarefa {  
    criarTarefa();  
  }  
}
```

- aula9\_throw1.dart

# catch

catch pode receber o erro como argumento.

```
void main() {  
  try {  
    adicionarTarefa();  
  } on SemTarefa catch(e) {  
    print('Ouve um erro: $e');  
    criarTarefa();  
  }  
}
```

- aula9\_throw2.dart

# finally

finally sempre será executado, com ou sem erro.

```
void main() {  
  try {  
    adicionarTarefa();  
  } on SemTarefa catch(e) {  
    print('Ouve um erro: $e');  
    criarTarefa();  
  } finally {  
    terminarTarefas();  
  }  
}
```

- aula9 throw3.dart

# Programação assíncrona

O programa continua enquanto aguarda uma operação terminar.

# Operações assíncronas

- Buscar dados da internet
- Ler e escrever arquivos
- Acessar banco de dados

# Resultado

O resultado de uma operação assíncrona é:

- `Future`
- `Stream` (se tiver várias partes)

# A programação assíncrona usa:

- Future
- async
- await

# Exemplo imediato

```
String reservaMesa() {  
    var mesa = buscaMesa();  
    return 'Sua mesa é: $mesa';  
}
```

```
String buscaMesa() {  
    return '7';  
}
```

```
void main() {  
    print(reservaMesa());  
}
```

# Prática

Altere o método `buscaMesa()` para retornar na String um número aleatório entre 1 e 10.

# Solução

```
String buscaMesa() {  
    var aleatorio = Random().nextInt(10) + 1;  
    return '$aleatorio';  
}
```

# Exemplo com atraso sem espera

```
import 'dart:math';

String reservaMesa() {
  var mesa = buscaMesa(); // falta esperar pelo futuro!
  return 'Sua mesa é: $mesa';
}

Future<String> buscaMesa() {
  return Future.delayed(const Duration(seconds: 2), () {
    var aleatorio = Random().nextInt(10) + 1;
    return '$aleatorio';
  }));
}

void main() {
  print(reservaMesa());
}
```

# Future

Uma Future é o resultado de uma operação assíncrona. Possui dois estados:

- uncompleted
- completed

# Usando future

Se usar future uncompleted: Future<T>

Se usar future completed: T ou Error

# Exemplo com espera

```
import 'dart:math';

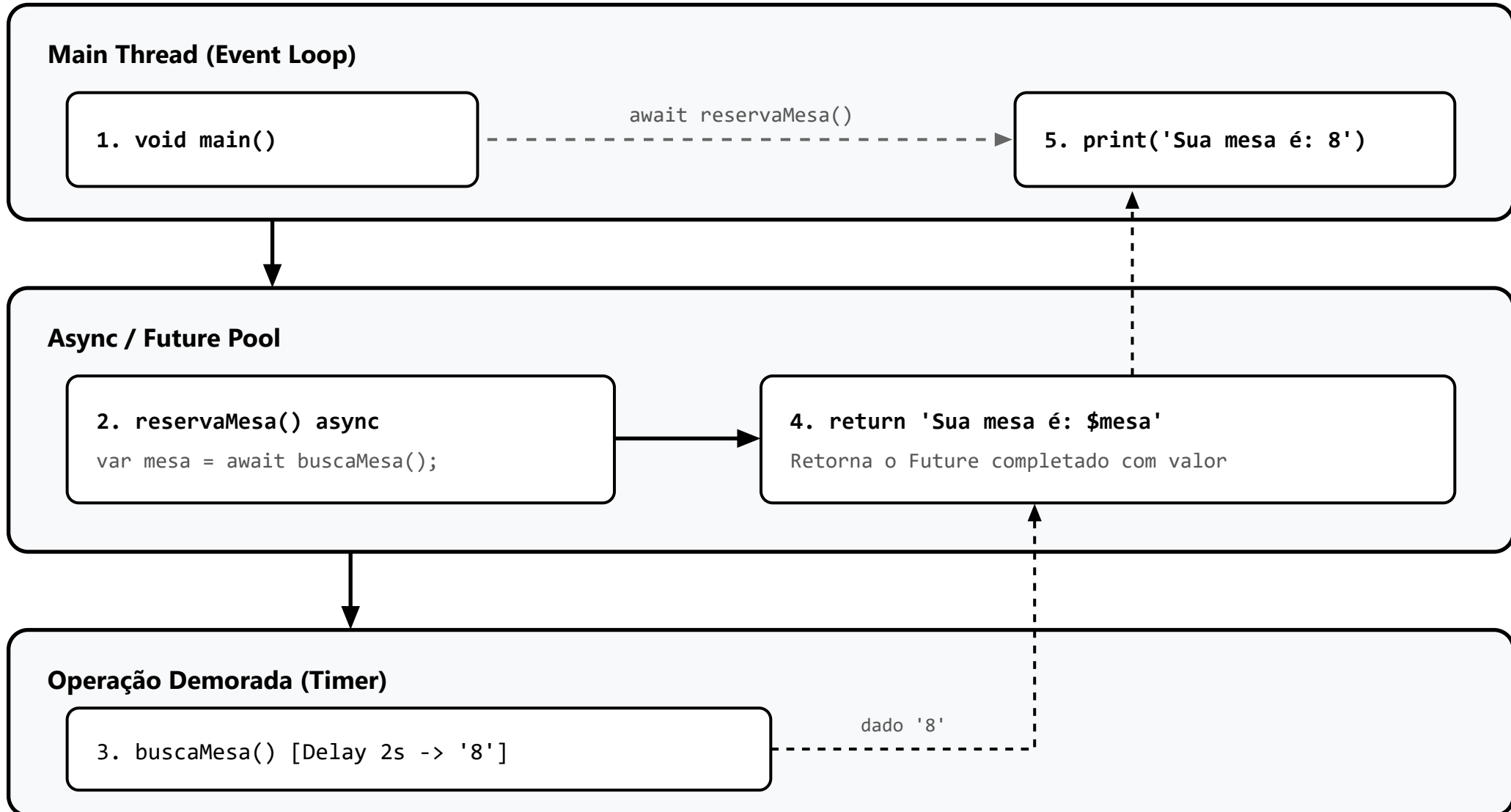
Future<String> reservaMesa() async {
  var mesa = await buscaMesa();
  return 'Sua mesa é: $mesa';
}

Future<String> buscaMesa() {
  return Future.delayed(const Duration(seconds: 2), () {
    var aleatorio = Random().nextInt(10) + 1;
    return '$aleatorio';
  }));
}

void main() async {
  print(await reservaMesa());
}
```

# Fluxo Assíncrono: reservaMesa()

`void main() async -> reservaMesa() -> buscaMesa()`



# Prática

Altere o método `buscaMesa()` para retornar na String um número aleatório entre 1 e 25.